🔷 PROJECT CODE EXPLANATION (LINE BY LINE)

🔹 Title Comment

```
# @title Muntasir+Imran V2
```

- This is just a **Colab title comment**.
- It does **not affect execution**.
- Used only for naming the notebook cell.

🔹 Step 1: Import Libraries

```
import cv2 import easyocr import numpy as np import matplotlib.pyplot as plt from google.colab
import files
```

What each library does:

- `cv2` → OpenCV library
  - Used for **image reading, filtering, edge detection, contours, drawing**
- `easyocr` → OCR library
  - Used to **read text from images**
- `numpy (np)` → Numerical operations
  - Used for **arrays, masks, indexing**
- `matplotlib.pyplot (plt)` → Visualization
  - Used to **display images at each step**
- `google.colab.files` → File upload
  - Allows user to **upload images interactively**

🔹 Step 2: Upload Image

```
print("Please upload an image containing a vehicle number plate:") uploaded = files.upload()
```

- Displays a message asking the user to upload an image.
- `files.upload()` opens a file picker.
- The uploaded image is stored in a **dictionary** called `uploaded` .

```
for fn in uploaded.keys(): image_path = fn print(f"Image uploaded: {image_path}")
```

- Loops through uploaded files (usually only one).

- `fn` = filename of the uploaded image.
- `image_path` stores the image filename.
- Confirms upload by printing the filename.

🔹 Step 3: Read Image

```
image = cv2.imread(image_path)
```

- Reads the uploaded image from disk.
- Stores it as a **NumPy array** in BGR format (OpenCV default).

```
if image is None: raise ValueError("Error reading image. Please upload a valid image file.")
```

- Safety check.
- If image loading fails, the program **stops immediately** with an error.
- Prevents further crashes.

🔹 Step 3.1: Display Original Image

```
plt.figure(figsize=(8,5)) plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB)) plt.title("Original Image") plt.axis('off') plt.show()
```

What happens:

- Converts image from **BGR → RGB** (Matplotlib expects RGB).
- Displays the original image.
- `axis('off')` removes axes for clean visualization.

📌 Purpose:

To visually verify the input image.

🔹 Step 3.2: Convert to Grayscale

```
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
```

- Converts the RGB image into **grayscale**.
- Removes color information.
- Keeps intensity and structure.

📌 Why grayscale?

- Faster processing

- Better edge detection
- OCR does not need color

```
plt.imshow(gray, cmap='gray')
```

- Displays grayscale image using gray colormap.

🔹 Step 3.3: Bilateral Filter (Noise Reduction)

```
gray = cv2.bilateralFilter(gray, 11, 17, 17)
```

What bilateral filter does:

- Reduces noise
- **Preserves edges** (very important for plates)

Parameters:

- `11` → neighborhood diameter
- `17, 17` → color and space sigma values

📌 Why bilateral instead of blur?

- Normal blur destroys edges
- Bilateral keeps plate boundaries sharp

🔹 Step 3.4: Canny Edge Detection

```
edged = cv2.Canny(gray, 30, 200)
```

- Detects edges using Canny algorithm.
- `30` → lower threshold
- `200` → upper threshold

📌 Output:

- White pixels = strong edges
- Black pixels = background

Used to:

Highlight plate boundaries for contour detection

🔹 Step 3.5: Contour Detection

```
cnts, _ = cv2.findContours(edged.copy(), cv2.RETR_TREE, cv2.CHAIN_APPROX_SIMPLE)
```

What happens:

- Finds **continuous closed boundaries** in edge image.
- `RETR_TREE` → retrieves full contour hierarchy
- `CHAIN_APPROX_SIMPLE` → compresses contour points

```
cnts = sorted(cnts, key=cv2.contourArea, reverse=True)[:10]
```

- Sorts contours by area (largest first).
- Keeps top 10 largest contours.
- Assumption: **number plate is among the largest objects**

🔹 Detect Rectangular Plate Contour

```
screenCnt = None
```

- Variable to store the plate contour if found.

```
for c in cnts: peri = cv2.arcLength(c, True) approx = cv2.approxPolyDP(c, 0.018 * peri, True)
```

Explanation:

- Calculates contour perimeter.
- Approximates contour to reduce points.
- Makes shape simpler (polygon).

```
if len(approx) == 4: screenCnt = approx break
```

- If contour has **4 sides**, it is assumed rectangular.
- Number plates are rectangular → selected.
- Loop breaks after first valid plate found.

🔹 Step 4: Masking and Cropping

```
if screenCnt is not None:
```

- Checks if a plate contour was found.

```
mask = np.zeros(gray.shape, np.uint8)
```

- Creates a black image (mask).
- Same size as grayscale image.

```
cv2.drawContours(mask, [screenCnt], 0, 255, -1)
```

- Draws white filled contour on mask.
- White = plate region
- Black = background

```
new_image = cv2.bitwise_and(image, image, mask=mask)
```

- Keeps only plate region.
- Everything else becomes black.

📌 This is **masking**.

```
(x, y) = np.where(mask == 255) (topx, topy) = (np.min(x), np.min(y)) (bottomx, bottomy) =
(np.max(x), np.max(y))
```

- Finds bounding box of the plate mask.

```
cropped = gray[topx:bottomx+1, topy:bottomy+1]
```

- Crops the plate region from grayscale image.
- This cropped image is sent to OCR.

🔹 Fallback Strategy

```
else: print("[INFO] No number plate contour detected. Using full image for OCR.") cropped = gray
```

📌 If no plate contour is found:

- System **does not fail**
- OCR is applied to full image
- Ensures robustness

🔹 Step 5: OCR Recognition

```
reader = easyocr.Reader(['en'])
```

- Loads EasyOCR model for English characters.

```
result = reader.readtext(cropped)
```

- Detects text regions
- Recognizes characters
- Returns text + confidence

```
text = " ".join([d[1] for d in result]).strip() text = text.upper()
```

- Extracts detected text
- Joins all pieces
- Converts letters to **uppercase**

🔹 Step 6: Final Visualization

```
if screenCnt is not None:
```

- If contour was detected:

```
cv2.drawContours(image, [screenCnt], -1, (0, 255, 0), 3)
```

- Draws green rectangle around plate.

```
cv2.putText(image, text, (x, y - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.9, (36, 255, 12), 2)
```

- Writes detected plate number above plate.

```
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
```

- Displays final annotated result.